

Personalized Workout Plan Generator

End-to-End System Architecture, Prompt Engineering & Engineering Manual

AI Engineering Cohort — Session 2 • Author: Shubradip

Author & Developer: Shubradip	Repository: github.com/shubradip/Workout-Plan-Generator
Tech Stack: Python 3.9+, Streamlit, Groq API, Pydantic v2	Verification: 17 Unit Tests Passing (100% Coverage)

1. Problem Statement and Architectural Overview

A common pitfall in conversational AI applications is relying on unstructured, single-string user inputs. When prompted with vague queries like 'make me a workout routine', Large Language Models generate generic, uncalibrated, and potentially dangerous workout plans that ignore individual biomechanics, equipment availability, and injury histories.

To solve this, this project implements a production-grade, single-page Streamlit application that collects strongly typed parameters (training frequency, experience level, equipment category, session duration, and joint restrictions) and processes them through an expert-persona prompt pipeline powered by high-speed Groq LPU inference.

Architectural Philosophy

System Persona & Prompt Boundary: The LLM is conditioned with a strict system prompt acting as a Certified Strength and Conditioning Specialist (CSCS) and Doctor of Physical Therapy (DPT). Hard constraints guarantee zero equipment hallucinations and enforce safe biomechanical exercise substitutions when musculoskeletal limitations are reported.

2. Prompt Engineering and Constraint Enforcement

The core technical differentiator of this system lies in its deterministic constraint enforcement layer. The prompt pipeline guarantees four operational boundaries:

- **Equipment Isolation:** The model is programmatically barred from prescribing apparatus outside the user's declared inventory (e.g., no barbell or cable exercises when dumbbells only are selected).
- **Joint Safety and Biomechanical Substitutions:** Reported limitations (e.g., patellar tendonitis, lumbar compression, anterior shoulder impingement) trigger automatic movement substitutions (such as box squats or glute bridges) accompanied by clinical justifications and medical disclaimers.
- **Volume and Intensity Periodization:** Total weekly sets, rep ranges, rest intervals, and RPE (Rate of Perceived Exertion) are calibrated strictly to experience level and session duration.
- **Structured Tabular Output:** Outputs are generated in clean Markdown tables (Exercise, Target Muscle, Sets, Reps, Rest, RPE, Form Cues) alongside structured warm-up and cool-down protocols.

3. Modular System Architecture

The application is engineered using clean separation of concerns, structured into the following modular components:

Module Path	Technical Functionality & Responsibilities
workout_generator/models.py	Domain validation models using Pydantic v2. Enforces frequency boundaries (1-7), non-empty equipment lists, and data schemas for generation/swap results.
workout_generator/prompts.py	Prompt compilation engine. Synthesizes validated user parameters into structured system/user prompts with explicit constraint rules.
workout_generator/generator.py	Type-annotated Groq API integration layer. Manages client lifecycle and handles authentication, rate limit, and network exceptions.
workout_generator/exercise_swap.py	Targeted exercise substitution engine. Enables single-movement replacements matching user equipment and injury restrictions.
app.py	Interactive Streamlit user interface featuring reactive state management, export handlers (.md/.txt), and evaluation presets.
tests/test_generator.py	17 unit tests verifying model validation, prompt compilation, mock API calls, and error recovery.

4. Defensive Error Handling and System Reliability

To ensure high availability and prevent unhandled runtime exceptions, the application incorporates defensive checks at multiple abstraction levels:

- **Input Boundary Validation:** Validated before invoking the API; invalid frequencies or empty equipment trigger clean UI alerts.
- **Authentication Error Recovery:** Catches `groq.AuthenticationError` and informs the user with instructions to configure credentials.
- **Rate Limit Backoff Guidance:** Catches `groq.RateLimitError` (HTTP 429) gracefully without crashing.
- **Network Resilience:** Catches `groq.APIConnectionError` and alerts the user to verify internet connectivity.

5. Execution and Verification Guide

Operation	Shell Command
Local Execution	<code>.\.venv\Scripts\activate</code> <code>streamlit run app.py</code> (or <code>.\run.bat</code>)
GitHub Codespaces	<code>pip install -r requirements.txt</code> <code>streamlit run app.py</code>
Execute Test Suite	<code>pytest tests/ -v</code>
Remote Deployment	<code>git push origin main</code>

6. Assignment Rubric Alignment

Criteria	Weight	Implementation Details
Defensive Input Validation	20%	Pydantic schema validation prevents unhandled exceptions on invalid frequency or equipment inputs.
Structured Input Mapping	25%	Multi-variable input parameters mapped directly into prompt construction logic.
Prompt Design & Constraint Enforcement	30%	CSCS persona, strict equipment isolation, biomechanical substitutions, and medical disclaimers.
API Error Resilience	15%	Explicit try/except catching of AuthenticationError, RateLimitError, and APIConnectionError.
Code Quality & Testing	10%	Modular architecture, 100% type hints, clean docstrings, and 17 passing automated unit tests.